Frontline

Scaling strategy for a creator-driven traffic spike

Written 2026-10-20 from the repository and dev-docs. Prices and plan limits come from memory of vendor pricing pages and are marked **verify**: confirm on each vendor's current pricing page and dashboard before spending money or relying on a number. Traffic figures are assumptions, not measurements.

1. Summary

Frontline has 219 users today. A partnership with a New York creator focused on trash and stewardship (almost 200k followers) could bring thousands of visitors within hours of a post. The current stack (Vercel, Supabase Pro, Railway Hobby, Cloudflare R2) can plausibly absorb a moderate spike, but it has **no safety margin and several unprotected weak points**. Nothing here needs a rewrite. The work is mostly plan upgrades, caching, rate limiting, connection settings, and rehearsal.

Recommended order of work:

- **Before any announcement:** upgrade Vercel to Pro and Railway to Pro, ship the auth and rate-limit lockdown, confirm Supabase signup email works at volume, add cache headers to the tile and stats endpoints, set up alerts, and run a load test against staging.
- **Ask the creator for lead time:** a heads-up of at least two weeks, a link to a stable landing page, and a posting time you know in advance so someone is watching.
- **Day of:** watch dashboards, keep a rollback path, and have a plan to turn off expensive features (maps heavy views, geo stats) if the database struggles.
- **After:** keep the upgrades that proved necessary, and downgrade nothing for at least a month.

2. What traffic to plan for

The honest answer is that nobody can predict this. Creator posts convert at very different rates, so plan for three scenarios and decide in advance what you do at each. All numbers are assumptions.

Scenario	Assumption	Visitors	Signups	Peak concurrent users
A. Modest	0.5 to 1% of followers click once	1,000 to 2,000 over 2 days	200 to 500	20 to 50
B. Strong	3 to 5% click, mention is pinned or in a video	6,000 to 10,000 over 3 days	1,500 to 3,000	100 to 300
C. Viral	Reshared, picked up by local press or other creators	30,000 to 60,000 over 1 week	8,000 to 15,000	500 to 1,500

Traffic is front-loaded: expect roughly a third of a post's total visits in the first one to three hours. Most visitors will be on phones, arriving from Instagram, TikTok, or YouTube in-app browsers. Those browsers mostly reach the website, not the native apps, so **the website on Vercel is the front door** and the app stores are secondary. Plan to have the landing experience work well for a signed-out visitor on a weak connection.

Each page view fans out: the layout runs several Supabase Auth and profile lookups, and the campaign page then makes API and database calls plus a MapTiler style and tile load. A single visit is dozens of requests, so request counts matter more than visitor counts.

3. Layer-by-layer assessment

3.1 Vercel (frontend host)

- **Plan.** The notes list Vercel as Hobby (free). Hobby is intended for non-commercial use and has tight caps on bandwidth, function execution, and build minutes (verify current numbers). A creator partnership with a possible revenue layer (Plus, sponsorships) is commercial. **Move to Pro (about $20 per seat per month, verify).**
- **Pro includes** far higher bandwidth and function allowances, spend management (set a cap so a spike cannot create a surprise bill), and analytics. Turn on Speed Insights and Web Analytics for the launch window.
- **Server rendering cost.** The root layout awaits Supabase Auth and several queries on every navigation (native-app-load-time doc, section 2). Under load this is the largest per-visit cost. Prefer fixing it (cache or parallelize, skip for signed-out visitors) over buying capacity.
- **Images.** The app has 75 raw `<img>` tags and no `next/image` usage, so avatars and event photos load at full size straight from R2. That is good for Vercel bill (no image optimization quota used) but bad for mobile load time and R2 egress. Fix with resized thumbnails or Cloudflare image resizing rather than Vercel image optimization, which is metered.
- **Deploys.** Do not deploy during the spike window except for hotfixes. Vercel supports instant rollback to a previous deployment.

3.2 Railway (FastAPI backend and cron)

- **Plan and shape today.** Hobby, one replica of the API in us-east4, restart on failure up to 5 times, with 4 cron services. A single process is a single point of failure: one crash loop or a deploy during the spike means downtime.
- **Upgrade to Pro (about $20 per month plus usage, verify).** Pro raises per-service resource ceilings and allows multiple replicas. Hobby caps resources per service (verify) and has no replica support.
- **Add a second replica** in the same region once on Pro. FastAPI here is stateless except one thing: the in-process MVT tile cache (`_tile_cache`, max 2,000 entries) is per instance and empties on every deploy or restart. That is fine for correctness but means each replica warms its own cache. Put a CDN in front of tiles (section 4) and the problem goes away.
- **Workers.** The Dockerfile starts one uvicorn process with no `--workers` flag. One async process handles many concurrent requests, but CPU-bound work (geometry, Shapely, tile encoding) blocks the event loop. Either run 2 to 4 workers per replica or scale replicas. Size by CPU use under a load test, not by guess.
- **Database pool.** `create_async_engine` in `backend/app/db/database.py` sets no pool arguments, so SQLAlchemy defaults apply: pool of 5 plus 10 overflow, so up to 15 connections per process. With 2 replicas and 3 workers that is up to 90 connections, which competes with Supabase's own use (section 3.3). Set `pool_size`, `max_overflow`, and `pool_pre_ping=True` explicitly and connect through the Supabase pooler.
- **Crons.** Decay (6h), event expiry (30 min), and reminders are light today. Their endpoints are public (security-lockdown doc). Lock them before the launch so a crawler cannot trigger them repeatedly.
- **No auth on most routes and no rate limiting.** This is the biggest operational risk. At low traffic it is a security gap. In a spike (or if the creator's audience includes bad actors) it lets anyone create load on the most expensive endpoints. Ship the lockdown work first.

3.3 Supabase (Postgres, Auth, Realtime, Edge Function)

- **Compute size is the key unknown.** Pro includes a compute credit that covers the smallest instance (verify which size this project runs, shown under Project Settings, Compute and Disk). The smallest instances have shared CPU and about 1 GB of RAM and low connection limits. Upgrade to a Small or Medium instance (verify prices, roughly $15 to $60 per month more) for the launch. Compute changes cause a brief restart, so do this days before, not during.

- **Connections.** FastAPI uses a direct database URL (port 5432 in `.env.example`). Direct connections are limited by instance size (verify the number). Use the **Supavisor pooler**. Transaction mode (port 6543) allows far more clients but needs asyncpg's prepared-statement cache disabled (`statement_cache_size=0`; verify against current Supabase docs). Test this change in staging before prod, because it affects every query.
- **Auth.** Signup, login, and session refresh all hit Supabase Auth, and the Next.js layout calls `getUser()` three to four times per request. Under a spike that multiplies auth load. Reduce calls per request (one per request, pass the result down). Check the Auth rate limits in the dashboard (per-IP sign-in and signup limits, token refresh limits, verify). Mobile carrier NAT can put many users behind one IP.
- **Signup email.** Supabase's built-in email sender has a very low hourly cap (verify). Confirm the project sends auth email through custom SMTP (Resend) with a verified domain, and that the Resend plan allows thousands of emails per day (free tier is roughly 100 per day and 3,000 per month, verify). If confirmation emails stall, new users cannot get in and will leave.
- **Realtime.** The map and feed subscribe to up to 6 channels per user session (stat bar, campaign events, problem reports, territory claims, notifications, achievements). Pro allows a limited number of concurrent connections (about 500 included, verify; more with the spend cap lifted). A scenario B or C peak can exceed this. Mitigation: subscribe only on the pages that need it, drop the stat bar channel for signed-out visitors, and fall back to polling when connection is refused.
- **Query load.** There are no materialized views, and leaderboards, streaks, and geo stats aggregate over the whole `contributions` table per request (database performance doc). With a few thousand new users that table could grow a lot in days. The first fixes are cheap: composite indexes and the streak-weeks view. Do step 1 and step 2 of that plan before launch, and the views if the measurements say so.
- **Row-level security cost.** About 57 policies use `auth.uid()`. Wrap them as `(select auth.uid())` so Postgres evaluates it once per query instead of once per row (a documented Supabase performance tip, verify applicability). Check in the Performance Advisor.
- **Backups.** Pro has daily backups. Point-in-time recovery is a paid add-on (verify). With thousands of new signups it is worth enabling before the launch. Do a restore test on a scratch project once, so the recovery steps are known.
- **Read replica.** Available as an add-on and carries its own cost. Only worth it if reads dominate and measurements show the primary is the bottleneck. Not first-line.
- **Storage of the keepalive.** The twice-weekly keepalive workflow is unnecessary on Pro. It is harmless.

3.4 Everything else

Service	Concern	Action
MapTiler	Already crossed the free tier and is on a $30 per month plan. Every map view loads a style plus tiles. A spike can blow through the quota and either bill overage or break every map at once (verify what the plan does at quota).	Check the usage dashboard and plan quota now. Do the usage reduction plan items that are cheap (swap mini maps to free tiles, debounce autocomplete) before launch. Know the next tier price. Have a fallback style ready (OpenFreeMap or CartoDB).
Cloudflare R2	Presigned uploads go direct from phones to R2, so they do not load the backend. Free tier covers 10 GB storage and 1M writes per month (verify). Egress is free.	Low risk. Add a CDN custom domain for the bucket so photos cache at the edge. Resize large photos on upload.
Resend	Welcome, confirmation, and reminder email volume rises with signups. Free tier limits (verify).	Move to the Pro tier (about $20 per month, verify) for the launch month. Confirm SPF, DKIM, and DMARC.
Firebase FCM	Free and high capacity. The Edge Function sends one push per notification insert.	Low risk. Avoid broadcast notifications from a single transaction to thousands of users at once.

Service	Concern	Action
Sentry	Error volume can spike and exhaust the free event quota, hiding real errors.	Check plan quota. Lower the trace sample rate and add spike protection for the launch.
Stripe	Payments are hidden behind a setting today.	No scaling concern. Keep hidden until the load is stable.
GitHub Actions	Not in the request path.	None.
Cloudflare (DNS, CDN)	Not currently in front of the site or API. This is the cheapest way to absorb spikes and bots.	See section 4. Put the API behind Cloudflare with caching and rate limit rules.

4. The highest-leverage fixes

These ranked by effect per hour of work. Most are pre-requisites for the launch.

#	Fix	Why it matters	Effort
1	Auth and rate limits on the FastAPI routes; lock cron endpoints (already scoped)	Closes the largest abuse risk before attention arrives. Also prevents a bot from running up compute.	Medium, scoped in security-lockdown doc
2	Cache headers on tile and stats endpoints, and a CDN in front (Cloudflare proxy on the API domain, or a Vercel rewrite)	Tiles are identical for everyone. Today only an in-process dict and `max-age=86400` on tiles help. JSON endpoints like leaderboards send no `Cache-Control`. A 30 to 60 second shared cache turns thousands of requests into a handful.	Small to medium
3	Explicit DB pool settings and pooler connection	Avoids connection exhaustion, the most common failure when a small stack suddenly gets traffic.	Small, needs staging test
4	Reduce per-request auth lookups in the root layout; skip user queries for signed-out visitors	Cuts Supabase Auth load and Vercel function time on every page view.	Medium
5	Composite indexes on `contributions` and the streak-weeks view; replace the JavaScript sum of all cleanups with an aggregate	Stops the heaviest pages getting slower as data grows.	Medium, scoped in database performance doc
6	Static, cacheable landing page for signed-out visitors (campaign hero, live counters cached 30 to 60s)	The first visit from the creator's audience should be a fast, cached page that does not touch the database per request.	Small to medium
7	Upgrade Vercel and Railway to Pro, raise Supabase compute, set spend caps	Removes plan ceilings and commercial-use risk. Spend caps prevent surprise bills.	Small, costs money
8	Thumbnails for avatars and event photos	Mobile load time and bandwidth.	Medium
9	MapTiler reduction items and a fallback style	Protects the most visible feature from quota failure.	Small to medium, scoped

5. Upgrade path by scenario

	A. Modest (up to ~500 signups)	B. Strong (~3k signups)	C. Viral (~15k signups)
Vercel	Pro	Pro, spend cap raised, caching on landing	Pro, consider Enterprise-style limits only if bandwidth bills grow (verify)

	A. Modest (up to ~500 signups)	B. Strong (~3k signups)	C. Viral (~15k signups)
Railway	Pro, 1 replica, explicit pool	Pro, 2 replicas, 2 workers each	Pro, 3+ replicas, Cloudflare caching, rate limits tuned
Supabase	Pro, current compute if CPU stays under 50% in a load test	Small or Medium compute, pooler, PITR on	Medium or Large compute, read replica if measurements justify, Realtime limit raised
MapTiler	Current $30 plan if quota allows	Next tier, free tiles for mini maps	Mixed: free tiles for most views, MapTiler only on the main map
Email	Resend free or Pro	Resend Pro	Resend Pro or higher plus a daily send monitor
Rough extra monthly cost (verify)	$20 to $60	$100 to $250	$300 to $700+

Costs are rough sums of vendor list prices and should be checked. They are monthly and can be dialed back after the spike. The Vercel and Railway Pro upgrades are the only ones that are justified in every scenario. Everything above that is driven by measurement.

6. Pre-launch checklist

Done ?	Item	How to check
[ ]	Vercel and Railway on Pro, spend caps set	Dashboards
[ ]	Supabase compute size known and sized for the scenario; PITR decision made	Project Settings, Compute and Disk; Backups
[ ]	Auth email sends via custom SMTP; send a test signup and confirm arrival time	Auth settings; real test
[ ]	FastAPI auth and rate limits live; cron endpoints locked	Security-lockdown doc, curl tests
[ ]	DB pool explicit; pooler tested on staging	Staging run; connection count graph
[ ]	Cache headers and CDN on tiles and public stats	Response headers, CDN hit ratio
[ ]	Staging environment exists and mirrors prod settings	Security-lockdown doc, staging section
[ ]	Load test run against staging (see section 8) with results written down	k6 or Locust report
[ ]	Realtime concurrent limit known; fallback behavior tested	Dashboard Realtime inspector
[ ]	MapTiler quota checked, fallback style ready	MapTiler dashboard
[ ]	Resend plan, domain authentication, and quota checked	Resend dashboard
[ ]	Sentry quota, alert rules, and on-call recipients set	Sentry settings
[ ]	Uptime monitor on /, /campaigns/trash-war, and /health (every minute, alerts to phone)	BetterStack, UptimeRobot, or similar
[ ]	Backup restore rehearsed on a scratch Supabase project	Written steps and timing
[ ]	Signed-out landing page is fast on slow 4G in the Instagram in-app browser	Real device test
[ ]	No deploys scheduled during the first 72 hours after the post	Calendar
[ ]	Signing key backup for Android done (open todo) in case a native hotfix is needed	Memory notes

7. Standing up a staging environment

A staging environment is a second, separate copy of the full stack that mirrors production settings but holds no real user data. For this launch it is where you load test, rehearse the Supavisor pooler change, and verify the auth lockdown before it reaches real users. The design is already worked out in **security-lockdown-and-staging-2026-10-17.md** (section 4) and **staging-environment-scoping-2026-08-10.md**; this section summarizes it and adds what a load test needs.

7.1 Target layout

Layer	Production	Staging
Database and Auth	Supabase Pro project	Second Supabase project in the same Pro org (about one extra Micro instance, roughly $10 per month, verify). A free separate org pauses after inactivity, which defeats the purpose. Size it up temporarily for load tests.
Backend	Railway service, deploys from master	Railway **environment** named staging in the same project, deploys from a staging git branch, own variables.
Frontend	Vercel production from master	Vercel branch deployment for the staging branch with Preview-scoped env vars, on a domain such as staging.frontlinemaps.com.
Storage	Cloudflare R2 bucket	Separate bucket and credentials (or a prefix with a scoped key).
Email	Resend, real domain	Resend test key, or Supabase Inbucket-style capture. Never send to real addresses from staging.
Maps	MapTiler key	Separate key restricted to the staging domain, or the free fallback style, so load tests do not eat production quota.
Push, Stripe, Sentry	Live	Push off. Stripe test mode keys. Sentry as a separate environment tag.

7.2 Steps

- **1. Supabase staging project.** Create it, then apply all migrations with the CLI against the staging project ref (run by you, never against production). Confirm schema parity with a migration list diff.
- **2. Auth settings.** Set the site URL and redirect allow-list for the staging domain, configure custom SMTP, and add a Google OAuth client or redirect URI for staging. Use a separate client if possible so the production Android sign-in is untouched.
- **3. Git branch.** Create `staging` from master. Workflow becomes feature branch, PR to staging, verify, PR from staging to master.
- **4. Railway environment.** Create the staging environment, deploy from the staging branch, and set staging-only variables: DATABASE_URL, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, ENVIRONMENT=staging, a different ADMIN_API_SECRET, CRON_SECRET, CORS_ORIGINS for the staging domain, R2 variables. Decide whether to declare it in `.railway/railway.ts` or configure it by hand.
- **5. Vercel.** Add Preview-scoped variables for the staging branch (staging Supabase URL and anon key, staging backend URL, ADMIN_API_SECRET), assign the staging subdomain, and add noindex. Confirm backend-only pushes do not trigger pointless frontend builds.
- **6. Admin router guard.** The admin seed, wipe, and recompute routes are currently gated only by ENVIRONMENT not equal to production. A staging backend would expose them publicly. Mount them only with an explicit ENABLE_ADMIN_ROUTER variable and the admin secret header before the first staging deploy.
- **7. Data.** Seed through the Seeder registry and `POST /admin/seed`. For load tests, seed to a size larger than production (for example 10,000 users and several hundred thousand contributions) so you see query behavior at the scale you expect after the launch. Do not copy real user data into staging.

- **8. Secrets.** Every secret must differ from production: service-role key, JWT keys, admin secret, cron secret, R2 and Stripe keys.
- **9. Smoke test.** Sign up, log a contribution, view the map and leaderboard, upload a photo, receive the auth email, and confirm the cron endpoints require their secret.

7.3 Making staging useful for load testing

- Match production **plan tiers** where it matters. A load test on a smaller database than the one you will run at launch tells you little. Temporarily raise staging compute to the planned launch size for the test window, then lower it.
- Keep staging **in the same regions** as production (Railway us-east4, same Supabase region) so latency matches.
- Test against the **same pooler mode and pool settings** you intend to ship.
- Tell vendors if needed. Large synthetic Auth traffic can look like an attack, so check Supabase's and MapTiler's policies and ramp gradually.
- Use a spend cap or alert on staging too. A forgotten raised compute size is the most common staging cost leak.

7.4 Cost and time

Item	Estimate (verify)
Second Supabase project	About $10 per month at Micro; roughly $15 to $60 for the days it runs at a larger size for load tests
Railway staging environment	Low single-digit dollars per month idle, more under test; on Pro it uses the same usage billing
Vercel branch deployment	Included in the Pro plan, no extra seat
R2, Resend test, MapTiler fallback	Effectively free at test volumes
Setup effort	About one to two focused days for steps 1 to 6 and a smoke test; another half day to script seed data and the first load test
Ongoing upkeep	Apply each new migration to staging first, keep the staging branch rebased on master, rotate secrets with production

Staging is a prerequisite for most other items in this plan (the auth lockdown, the pooler change, the materialized views, and any load test), so build it first. It also pays off after the launch as the normal place to test payments v2 before enabling it.

8. Load testing

- **Never against prod.** Use the staging environment from section 7 (scoped, not yet built) or a local stack with a prod-sized dataset. Seeding must go through the Seeder registry and `POST /admin/seed`.
- **Tool.** k6 (JavaScript scripts, easy to run from a laptop) or Locust. A free tier of k6 Cloud is enough for the numbers here.
- **Scenarios to script.** (1) signed-out visitor loads the home page and campaign page, with 10 to 20 tile requests; (2) signup and login; (3) a logged-in user opens the campaign, logs a contribution, and views the leaderboard; (4) 200 users hold a Realtime connection.
- **Ramp.** Start at 20 virtual users, then 100, 300, and 1,000 for ten minutes each. Record p95 latency, error rate, API CPU and memory, Postgres CPU, active connections, and Auth request errors.
- **Pass criteria (suggested).** p95 under 1 second for API reads and under 3 seconds for first page load on a throttled connection, error rate under 1%, Postgres CPU under 70%, connections under 80% of the limit.

- **Upstream warning.** Check each vendor's load testing policy before testing against its services (Supabase Auth in particular), or throttle tests so they are not mistaken for an attack.

9. Monitoring and alerts for launch week

What	Where	Alert when
Request latency, error rate, function duration	Vercel Analytics and Logs	5xx above 1%, or p95 above 3s
API CPU, memory, restarts	Railway metrics	CPU above 80% for 5 min, any crash loop
DB CPU, memory, disk, connections	Supabase Reports and Query Performance	CPU above 70%, connections above 80%, slow queries rising
Auth signups, errors, rate-limit hits	Supabase Auth logs	Any 429 from Auth in normal flow
Realtime connections	Supabase Realtime report	Above 80% of limit
Map quota	MapTiler dashboard	Above 70% of monthly quota
Email	Resend dashboard	Bounce rate above 5%, daily cap near
Exceptions	Sentry	New issue with more than 20 events in 10 minutes
Uptime	External uptime monitor	Any downtime, check every minute

10. Launch-day runbook

- **T minus 7 days:** upgrades done, load test passed, creator's post time agreed, status plan written.
- **T minus 1 day:** freeze deploys, verify backups ran, confirm alert recipients, check quotas.
- **T zero to T plus 3 hours:** one person watches Vercel, Railway, Supabase, and Sentry dashboards the whole time. Post a short note in the app (banner) only if there is a problem.
- **If the database is saturated:** 1) raise the compute add-on (about a minute of restart, in a quiet moment); 2) lengthen cache times on stats and leaderboards; 3) turn off the geographic stats layers by default (already opt-in); 4) disable non-critical Realtime channels.
- **If the API is down or slow:** add replicas in Railway, check the pool and connection count, roll back the last deploy if one happened.
- **If maps fail:** switch the style URL to the free fallback with an environment variable and redeploy (this needs to be a one-line change; build that switch ahead of time).
- **If signup email stalls:** check Resend quota and Supabase SMTP settings; consider temporarily disabling email confirmation only if the product owner accepts the abuse tradeoff.
- **After:** write a short retro with peak numbers, what broke, and what cost what. Use it to set permanent plan sizes.

11. Risks that are not purely technical

- **Moderation.** More users means more photos, more problem reports, and more chances for bad content. Check that the admin moderation queue is usable at ten times today's volume, and that someone is assigned to it for launch week.
- **Data quality.** New users in New York, probably concentrated in the same few boroughs, will stress the geo features. The NYC borough data is still missing in prod (see the deferred note), so the Borough stats layer will not work for them until it is loaded. Decide whether to load it before launch.

- **First impression.** Visitors from a creator link decide in seconds. A fast signed-out landing page matters more than any backend scaling.
- **Native apps.** Store review takes days, so app-only fixes cannot be hotfixed. Keep app logic on the website side during the window. The apps load the live site, so web fixes still ship instantly.
- **Payments.** Keep the money features hidden until load is stable and the payments migrations (109 to 120) are applied and tested in prod. Adding a launch and a payments release in the same week doubles the risk.
- **Creator relationship.** Ask for a rollout in phases if possible (a story first, then a video later). It is the cheapest scaling lever that exists.

12. Open questions to answer before launch

- Which Supabase compute size and which Vercel plan are active right now? (Confirm in the dashboards.)
- When is the creator planning to post, and how many posts or mentions?
- What is the call to action: signup, a specific cleanup event, or a campaign page? A single destination is easier to cache and test.
- Is the staging environment going to be built before the launch? If not, where will load tests run?
- Who is on call on launch day, and how do they get alerted?